

# Aula 4 — Terminal e Processos em Background

---

## 1. O que é o Terminal (visão simples)

---

Até agora falamos de conceitos (processo, usuário, serviço). O **terminal** é a ferramenta com a qual você vai *conversar* com o sistema operacional digitando comandos, em vez de clicar em ícones. Ele é a porta de entrada para quase tudo que um analista de Blue Team faz no dia a dia: investigar um host comprometido raramente é feito só com o mouse.

O programa que interpreta os comandos que você digita chama-se **shell** (no Linux, o mais comum é o **Bash**). Terminal = a janela onde você digita; shell = o "intérprete" que roda dentro dela e realmente executa os comandos.

```
Você digita → Terminal (janela) → Shell/Bash (interpreta) → Sistema Operacional executa
```

**Por que importa para segurança:** praticamente todo log de "execução suspeita" que você vai investigar mostra um **comando de shell** — seja em um Linux comprometido, seja em um Windows via PowerShell (que tem seu próprio "shell", como veremos no Módulo 02). Saber ler uma linha de comando é uma habilidade central de qualquer investigação.

## 2. Foreground vs. Background

---

Quando você executa um comando normalmente, ele roda em **foreground** ("primeiro plano"): o terminal fica "preso" esperando aquele comando terminar antes de aceitar o próximo.

Um processo em **background** ("segundo plano") continua rodando, mas libera o terminal para você digitar outros comandos enquanto isso.

No Bash, você coloca algo em background adicionando `&` no final:

```
sleep 300 &
```

Isso inicia um comando que "dorme" por 300 segundos, mas devolve o terminal para você imediatamente. Para ver o que está rodando em background na sessão atual:

```
jobs
```

E para trazer de volta ao foreground:

```
fg
```

## 3. `nohup` — sobrevivendo ao fechamento do terminal

---

Por padrão, se você fecha o terminal (ou desconecta de uma sessão SSH), os processos em background que você iniciou também são encerrados — eles "pertencem" àquela sessão.

O comando `nohup` ("no hangup") resolve isso: faz o processo continuar rodando **mesmo depois que você sair**.

```
nohup ./meu_script.sh &
```

**Por que importa para segurança:** isso é exatamente o tipo de comando que aparece em investigações reais. Um atacante que ganhou acesso via SSH pode rodar algo com `nohup ... &` para garantir que o processo malicioso continue ativo mesmo depois que ele desconectar — uma forma simples de manter execução contínua sem precisar reiniciar o processo a cada acesso. Não é, por si só, uma técnica de persistência "oficial" do MITRE ATT&CK (persistência de verdade sobrevive a um *reboot* da máquina, o que `nohup` sozinho não garante — isso normalmente é feito via serviço/cron, que vimos na Aula 3 e veremos com comandos reais no Módulo 01), mas é um padrão de comportamento que vale a pena reconhecer: **um processo cujo "processo pai" já não existe mais** (porque a sessão SSH que o criou já fechou) é um dado interessante em uma investigação — indica que alguém deliberadamente desacoplou aquele processo da sessão que o criou.

## 4. Ver o que está rodando: `ps`, `top`, `jobs`

Rápida prévia (comandos completos no Módulo 01):

- `jobs` → processos em background **da sua sessão de terminal atual**.
- `ps -ef` → **todos** os processos do sistema, de todos os usuários (já usamos na Aula 1).
- `top` → uma visão "ao vivo", atualizada automaticamente, mostrando quais processos mais consomem CPU/memória agora.

## 5. Conectando com o que já vimos

```
Terminal → Shell (Bash) interpreta comando
├─ cria um Processo (Aula 1: PID, usuário, pai/filho)
│   └─ roda em foreground (prende o terminal) OU
│       └─ roda em background (& / nohup) → libera o terminal,
│           pode sobreviver ao fechamento da sessão
```

## 6. Exercício prático

```
sleep 60 &
jobs
ps -ef | grep sleep
```

Observe: 1. `jobs` mostra o processo `sleep` rodando em background, dentro da sua sessão. 2. `ps -ef | grep sleep` mostra a mesma informação, mas de um ângulo diferente: o `PID`, o usuário dono do processo, e o **processo pai** (a coluna `PPID` — o PID do processo que o iniciou, nesse caso o seu shell/Bash). Guarde essa ideia de PPID — ela vai aparecer o tempo todo quando começarmos a investigar árvores de processos suspeitas.

Não se preocupe em decorar as opções do `ps` agora — vamos detalhar tudo isso com calma no Módulo 01.

## 7. Recapitulando o Módulo 00

---

Com as Aulas 1-4, você já tem o vocabulário básico para começar a mexer de verdade em um terminal Linux:

- **Hardware** (CPU/RAM/disco) executa e guarda dados.
- **Processos** rodam "como" um usuário, com PID, pai/filho e command line.
- **SO** controla usuários/grupos/permisões (quem pode fazer o quê).
- **Serviços** são processos que ficam sempre ativos — alvo comum de persistência.
- **Cliente/servidor** define quem inicia uma conexão de rede.
- **VM/container** isolam ambientes, com níveis diferentes de força.
- **Terminal/shell** é como você comanda tudo isso; **foreground/ background** definem se um processo "prende" ou não sua sessão.

Módulo 00 concluído. 🎉

## Próximo módulo

---

**Módulo 01 — Linux para Blue Team:** vamos parar de só falar sobre os comandos e começar a executá-los de verdade — navegação no sistema de arquivos, permissões na prática, processos, logs e os comandos que um analista usa todos os dias.

## Fontes recomendadas

---

- **GNU Bash Manual — "Job Control"**: referência oficial sobre foreground/ background, `jobs`, `fg`, `bg`.  
<https://www.gnu.org/software/bash/manual/bash.html#Job-Control>
- **Linux man-pages — `nohup(1)`**: comportamento oficial do comando. <https://man7.org/linux/man-pages/man1/nohup.1.html>